TD4 - Preuves de correction

Mathématiques Discrètes - Correction détaillée

Licence 1 Informatique
CERI - Avignon Université
Hiver 2026

Introduction

L'objectif de ce TD est de pratiquer les preuves de correction d'algorithmes. Ce document présente les énoncés des exercices ainsi que leurs corrections détaillées.

Note importante

Concepts clés à retenir :

- **Correction partielle** : Si le programme termine, alors le résultat est correct
- **Correction totale** : Le programme termine ET le résultat est correct
- **Invariant de boucle** : Propriété vraie à chaque itération
- **Triplet de Hoare** : $\{P\}S\{Q\}$ où P est la précondition, S l'instruction, Q la post-condition

1 Procédure simple

1.1 Exercice 1

Considérez la séquence d'instructions suivante :

$$y \leftarrow 2 \tag{1}$$
$$x \leftarrow x + y \tag{2}$$
$$x \leftarrow y - 2 \tag{3}$$

1.1.1 Solution

Note importante

Correction : Il y a une erreur dans l'énoncé original. La troisième instruction devrait être $x \leftarrow y \cdot z$ où $z = x + y$. Nous allons corriger cela.

Séquence corrigée :

$$y \leftarrow 2 \tag{4}$$
$$z \leftarrow x + y \tag{5}$$
$$x \leftarrow y \cdot z \tag{6}$$

1. Assertion initiale $x = 5$ et assertion finale $x = 14$
Trace d'exécution :

1. $y \leftarrow 2 : y = 2$
2. $z \leftarrow x + y = 5 + 2 = 7 : z = 7$
3. $x \leftarrow y \cdot z = 2 \cdot 7 = 14 : x = 14$

Conclusion : L'assertion finale $x = 14$ est vérifiée.
2. Assertion initiale $x = -5$ et assertion finale $x = -6$
Trace d'exécution :

1. $y \leftarrow 2 : y = 2$
2. $z \leftarrow x + y = -5 + 2 = -3 : z = -3$
3. $x \leftarrow y \cdot z = 2 \cdot (-3) = -6 : x = -6$

Conclusion : L'assertion finale $x = -6$ est vérifiée.
3. Assertion initiale $x = -2$ et assertion finale $x = 0$
Trace d'exécution :

1. $y \leftarrow 2 : y = 2$
2. $z \leftarrow x + y = -2 + 2 = 0 : z = 0$
3. $x \leftarrow y \cdot z = 2 \cdot 0 = 0 : x = 0$

Conclusion : L'assertion finale $x = 0$ est vérifiée.

2 Procédures avec blocs conditionnels

2.1 Exercice 2

2.1.1 Solution

1. Correction partielle pour l'assertion initiale $\top$ et l'assertion finale $z \geq 0$

```
Si x < 0 Alors
    z     0
FinSi
```

Preuve par cas :
Cas 1 : $x < 0$

1. Précondition : $\top \wedge (x < 0)$
2. Instruction : $z \leftarrow 0$
3. Postcondition : $z = 0 \geq 0$

Cas 2 : $x \geq 0$

1. Précondition : $\top \wedge \neg(x < 0) = (x \geq 0)$
2. Aucune instruction exécutée
3. Postcondition : z conserve sa valeur initiale (pas de contrainte sur z initialement)

Note importante

Note importante : Cette preuve n'est correcte que si $z \geq 0$ initialement dans le cas où $x \geq 0$. L'énoncé devrait préciser la valeur initiale de z.

2. Correction partielle pour $\top$ et $(x \leq y \wedge \min = x) \vee (x > y \wedge \min = y)$

```
Si x < y Alors
    min     x
Sinon
    min     y
FinSi
```

Preuve par cas :
Cas 1 : $x < y$

1. Précondition : $\top \wedge (x < y)$
2. Instruction : $\min \leftarrow x$
3. Postcondition : $\min = x$ et $x < y$ implique $x \leq y$
4. Donc : $(x \leq y \wedge \min = x)$

Cas 2 : $x \geq y$

1. Précondition : $\top \wedge \neg(x < y) = (x \geq y)$
2. Instruction : $\min \leftarrow y$
3. Postcondition : $\min = y$ et $x \geq y$ implique $x > y$ (si $x \neq y$) ou $x = y$
4. Donc : $(x > y \wedge \min = y) \vee (x = y \wedge \min = y)$

3 Procédures avec boucles

3.1 Exercice 3.1 : Calcul de puissance

Considérons la procédure suivante :

```
Proc dure puissance(x : r el , n : entier strictement positif)
    puissance     1
    i     0
    TantQue i < n Faire
        puissance     puissance * x
        i     i + 1
    FinTantQue
    Renvoyer puissance
FinProc dure
```

3.1.1 Solution

1. Invariant de boucle
Invariant proposé : $I : (puissance = x^i) \wedge (0 \leq i \leq n)$
2. Vérification de l'invariant
Initialisation :

- Avant la boucle : $puissance = 1$, $i = 0$
- $x^0 = 1$ et $0 \leq 0 \leq n$

Conservation :

- Supposons I vrai avant une itération : $(puissance = x^i) \wedge (0 \leq i < n)$
- Après l'itération :
  - $puissance_{nouveau} = puissance_{ancien} \times x = x^i \times x = x^{i+1}$
  - $i_{nouveau} = i_{ancien} + 1$
- Donc : $puissance_{nouveau} = x^{i_{nouveau}}$ et $1 \leq i_{nouveau} \leq n$

3. Terminaison
Variant de boucle : $V = n - i$

- V est un entier
- $V > 0$ tant que $i < n$ (condition de boucle)
- V décroît strictement à chaque itération (car i augmente de 1)

4. Correction totale
À la sortie de la boucle :

- L'invariant est vrai : $puissance = x^i$
- La condition de boucle est fausse : $i \geq n$
- Combiné avec l'invariant $i \leq n$, on a $i = n$
- Donc : $puissance = x^n$

3.2 Exercice 3.2 : Division entière

```
Proc dure division(a, d : entiers strictement positifs)
    r     a
    q     0
    TantQue r     d Faire
        r     r - d
        q     q + 1
    FinTantQue
    Renvoyer q, r
FinProc dure
```

3.2.1 Solution

1. Invariant de boucle
Invariant proposé : $I : (a = d \times q + r) \wedge (r \geq 0)$
2. Vérification de l'invariant
Initialisation :

- Avant la boucle : $r = a$, $q = 0$
- $a = d \times 0 + a$ et $a > 0$ (hypothèse)

Conservation :

- Supposons I vrai avant une itération et $r \geq d$
- Après l'itération :
  - $r_{nouveau} = r_{ancien} - d$
  - $q_{nouveau} = q_{ancien} + 1$
- Vérifions l'invariant :

$$\begin{aligned} a &= d \times q_{ancien} + r_{ancien} \text{ (hypothèse)} && (7)\\ &= d \times q_{ancien} + (r_{nouveau} + d) && (8)\\ &= d \times (q_{ancien} + 1) + r_{nouveau} && (9)\\ &= d \times q_{nouveau} + r_{nouveau} && (10) \end{aligned}$$

- Et $r_{nouveau} = r_{ancien} - d \geq 0$ car $r_{ancien} \geq d$

3. Terminaison
Variant de boucle : $V = r$

- $V \geq 0$ (par l'invariant)
- V décroît strictement à chaque itération (car r diminue de $d > 0$)
- La boucle se termine quand $r < d$

4. Correction totale
À la sortie :

- L'invariant est vrai : $a = d \times q + r$ et $r \geq 0$
- La condition de boucle est fausse : $r < d$
- Donc : $0 \leq r < d$

4 Procédures récursives

4.1 Exercice 4

4.1.1 Solution

1. Procédure somme(n) calcule $\sum_{k=0}^{n} k$

```
Proc dure somme(n : entier positif)
    Si n = 0 Alors
        Renvoyer 0
    Sinon
        Renvoyer n + somme(n - 1)
    FinSi
FinProc dure
```

Preuve par induction :
Cas de base : $n = 0$

- $somme(0) = 0$ (par définition)
- $\sum_{k=0}^{0} k = 0$

Cas récursif : Supposons $somme(n-1) = \sum_{k=0}^{n-1} k$ pour $n > 0$

$$\begin{aligned} somme(n) &= n + somme(n-1) \text{ (par définition)} && (11)\\ &= n + \sum_{k=0}^{n-1} k \text{ (hypothèse d'induction)} && (12)\\ &= \sum_{k=0}^{n} k && (13) \end{aligned}$$

2. Procédure factorielle(n) calcule $n!$

```
Proc dure factorielle(n : entier positif)
    Si n = 0 Alors
        Renvoyer 1
    Sinon
        Renvoyer n * factorielle(n - 1)
    FinSi
FinProc dure
```

Preuve par induction :
Cas de base : $n = 0$

- $factorielle(0) = 1$ (par définition)
- $0! = 1$ par convention

Cas récursif : Supposons $factorielle(n-1) = (n-1)!$ pour $n > 0$

$$\begin{aligned} factorielle(n) &= n \times factorielle(n-1) \text{ (par définition)} && (14)\\ &= n \times (n-1)! \text{ (hypothèse d'induction)} && (15)\\ &= n! && (16) \end{aligned}$$

5 Analyser des procédures

5.1 Exercice 5.1 : Procédure $p_{5,1}$

```
Proc dure  p  ,  (n : entier strictement positif, m : entier)
    Si n = 1 Alors
        Renvoyer m
    Sinon
        Renvoyer m +  p  ,  (n - 1, m)
    FinSi
FinProc dure
```

5.1.1 Solution

1. Résultat pour $n = 2$ et $m = 2$
Trace d'exécution :

1. $p_{5,1}(2, 2)$: $n \neq 1$, donc renvoie $2 + p_{5,1}(1, 2)$
2. $p_{5,1}(1, 2)$: $n = 1$, donc renvoie 2
3. Résultat final : $2 + 2 = 4$

2. Résultat pour $n = 3$ et $m = 0$
Trace d'exécution :

1. $p_{5,1}(3, 0)$: renvoie $0 + p_{5,1}(2, 0)$
2. $p_{5,1}(2, 0)$: renvoie $0 + p_{5,1}(1, 0)$
3. $p_{5,1}(1, 0)$: renvoie 0
4. Résultat final : $0 + 0 + 0 = 0$

3. Résultat pour $n = 3$ et $m = 4$
Trace d'exécution :

1. $p_{5,1}(3, 4)$: renvoie $4 + p_{5,1}(2, 4)$
2. $p_{5,1}(2, 4)$: renvoie $4 + p_{5,1}(1, 4)$
3. $p_{5,1}(1, 4)$: renvoie 4
4. Résultat final : $4 + 4 + 4 = 12$

4. Rôle de la procédure
D'après les résultats :

— $p_{5,1}(2, 2) = 4 = 2 \times 2$
— $p_{5,1}(3, 0) = 0 = 3 \times 0$
— $p_{5,1}(3, 4) = 12 = 3 \times 4$

Conclusion : La procédure $p_{5,1}(n, m)$ calcule $n \times m$ (multiplication).
5. Preuve de correction
Preuve par induction sur n :
Cas de base : $n = 1$

— $p_{5,1}(1, m) = m = 1 \times m$

Cas récursif : Supposons $p_{5,1}(k, m) = k \times m$ pour $k \geq 1$

$$p_{5,1}(k + 1, m) = m + p_{5,1}(k, m) \text{ (par définition)} \tag{17}$$
$$= m + k \times m \text{ (hypothèse d'induction)} \tag{18}$$
$$= (k + 1) \times m \tag{19}$$

5.2 Exercice 5.2 : Procédure $p_{5,2}$

```
Proc dure  p  ,   (x : r el)
    y     2
    z     x + y
    Si z > 0 Alors
        z     3
    Sinon
        z     0
    FinSi
    Renvoyer z
FinProc dure
```

5.2.1 Solution

1. Résultat pour $x = 3$

1. $y \leftarrow 2$
2. $z \leftarrow 3 + 2 = 5$
3. $z > 0$? Oui, donc $z \leftarrow 3$
4. Résultat : 3

2. Résultat pour $x = -2$

1. $y \leftarrow 2$
2. $z \leftarrow -2 + 2 = 0$
3. $z > 0$? Non, donc $z \leftarrow 0$
4. Résultat : 0

3. Résultat pour $x = -4$

1. $y \leftarrow 2$
2. $z \leftarrow -4 + 2 = -2$
3. $z > 0$? Non, donc $z \leftarrow 0$
4. Résultat : 0

4. Rôle de la procédure

D'après les résultats :

— Si $x + 2 > 0$ (i.e., $x > -2$) : résultat $= 3$

— Si $x + 2 \leq 0$ (i.e., $x \leq -2$) : résultat $= 0$

Conclusion : $p_{5,2}(x) = \begin{cases} 3 & \text{si } x > -2 \\ 0 & \text{si } x \leq -2 \end{cases}$

5. Correction de la procédure

La procédure est correcte par construction : elle implémente exactement la fonction définie ci-dessus par analyse des cas.

Conseils pour l'examen

Note importante

Méthodologie générale :

1. **Traces d'exécution :** Toujours suivre pas à pas l'algorithme
2. **Invariants :** Chercher des propriétés qui restent vraies à chaque itération
3. **Variants :** Pour la terminaison, trouver une quantité qui décroît strictement
4. **Induction :** Pour les algorithmes récursifs, utiliser le principe d'induction

Types de correction :

- **Correction partielle :** $\{P\}S\{Q\}$ - Si S termine avec P vraie, alors Q est vraie
- **Correction totale :** $[P]S[Q]$ - S termine ET si P vraie alors Q vraie
- **Invariant de boucle :** Propriété I telle que $\{I \wedge condition\}corps\{I\}$
- **Variant de boucle :** Expression entière positive qui décroît à chaque itération

Erreurs courantes à éviter :

- Oublier de vérifier l'initialisation de l'invariant
- Confondre correction partielle et totale
- Ne pas prouver la terminaison (variant manquant)
- Invariant trop faible ou trop fort